

SIMATS ENGINEERING

CO3_ASSESSMENT TOOL3_ Resolution Algorithm Implementation

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192511053
Name	B Santhosh Kumar

COURSE OUTCOME COVERED

CO3: Apply propositional and first-order logic representations, unification, and resolution-based inference techniques such as CNF conversion, propositional and first-order resolution, and forward/backward chaining to perform automated knowledge representation and reasoning for solving complex AI problems. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Q1.

Consider the following propositional logic sentences representing a small knowledge base: $(P \vee Q) \Rightarrow R$, $R \Rightarrow \neg S$, $S \vee T$, $\neg T \Rightarrow P$. (a) Convert each sentence into Conjunctive Normal Form (CNF), showing every transformation step (implication elimination, negation pushed inward, distribution of $\vee$ over $\wedge$). (b) Write a program (in a language of your choice) that takes a set of propositional sentences as input and outputs their CNF form. Test your program on the sentences above and present the output.

Q2.

Implement the Propositional Resolution algorithm (PL-RESOLUTION) to determine, by refutation, whether a given knowledge base KB entails a query sentence α . Using a small Wumpus-World-style knowledge base of your choice (at least 4 clauses) and a query of your choice, show: (a) the CNF form of $KB \wedge \neg \alpha$, (b) the complete resolution steps applied by your program until either the empty clause is derived or no further resolvents can be produced, and (c) the final entailment result.

Q3.

Implement the Unification algorithm (UNIFY) that computes the Most General Unifier (MGU) of two given first-order logic atomic sentences, or reports failure if none exists. Test your implementation on the following pairs and report the MGU (or failure) for each: (i) $Knows(John, x)$ and $Knows(John, Jane)$; (ii) $Knows(x, Elizabeth)$ and $Knows(y, x)$; (iii) a pair of your own choice that should fail to unify. Explain, with reference to the occurs check, why the third pair fails.

Q4.

Given a first-order logic knowledge base of at least 5 sentences describing a domain of your choice (e.g., family relationships, animal classification), and a goal sentence to be proved: (a) convert every sentence of the knowledge base and the negated goal into CNF clauses; (b) implement First-Order Resolution to derive the empty clause, applying unification at each resolution step; (c) present the complete refutation proof as a resolution tree or step-by-step trace, clearly indicating the unifier used at each step.

Q5.

Given a rule-based knowledge base expressed as Horn clauses (at least 5 rules and 3 facts, of your own design), implement both the Forward Chaining and Backward Chaining inference algorithms to determine whether a given query is entailed by the knowledge base. (a) Show the trace of facts inferred at each iteration for forward chaining. (b) Show the AND-OR proof tree / goal-reduction trace for backward chaining. (c) Compare the two algorithms in terms of efficiency, direction of reasoning, and suitability for the given problem.

Code of Conduct:

*I **B Santhosh Kumar/192511053** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

B Santhosh Kumar

Signature of the student:**RUBRICS FOR CALCULATION**

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
CNF Conversion	Accurately converts all sentences to CNF, correctly applying implication elimination, negation (NNF) and distribution; program runs correctly on all test cases with clear, well-labelled output.	Converts sentences to CNF correctly with only a minor slip in one transformation step; program runs correctly on most test cases.	Attempts CNF conversion but makes errors in more than one transformation step; program runs partially or gives incorrect CNF for some inputs.	Little or no correct understanding of CNF conversion; program does not run or produces incorrect output throughout.	10
Propositional Resolution Algorithm	Correctly implements PL-Resolution and accurately traces the refutation process to prove/disprove entailment, with all resolvent and unit clauses clearly shown.	Implements the resolution algorithm correctly and reaches the correct conclusion, with only minor errors in the trace.	Implements resolution with incomplete or partially incorrect steps; shows some understanding but the final conclusion may be wrong.	Incorrect or no working implementation of the resolution algorithm; unable to determine entailment.	10

Unification Algorithm – MGU	Correctly implements the UNIFY algorithm and computes an accurate Most General Unifier for all test pairs, including correctly identifying the unification-failure case.	Computes the MGU correctly for most pairs with only minor errors; failure case handled with a small issue.	Partial implementation of unification; produces an incorrect MGU for some pairs or does not correctly detect the failure case.	Little or no correct implementation of the unification algorithm.	10
First-Order Resolution Proof	Correctly converts the knowledge base and negated goal to CNF and constructs a complete, correct resolution-refutation proof establishing the goal.	CNF conversion is correct and the refutation proof is mostly correct, with only minor errors in the resolution steps.	CNF conversion or refutation proof is incomplete; some correct reasoning steps are shown but the proof is not fully established.	Incorrect or missing CNF conversion and resolution proof.	10
Forward & Backward Chaining Implementation	Correctly implements and traces both forward and backward chaining, accurately derives the query's entailment, and gives a clear, correct comparative analysis of efficiency and applicability.	Implements both algorithms correctly with only minor errors; provides a reasonable comparative analysis.	Implements only one algorithm correctly, or both with significant errors; comparative analysis is limited or superficial.	Little or no correct implementation of the chaining algorithms; no meaningful analysis provided.	10

Q1. CNF CONVERSION AND CNF PROGRAM

Aim

To convert propositional logic sentences into Conjunctive Normal Form (CNF) and implement a Python program for CNF conversion.

Given Knowledge Base

1. $(P \vee Q) \Rightarrow R$
2. $R \Rightarrow \neg S$
3. $S \vee T$
4. $\neg T \Rightarrow P$

(a) CNF Conversion

Sentence 1

$$(P \vee Q) \Rightarrow R$$

Implication elimination:

$$(P \vee Q) \Rightarrow R$$

$$= \neg(P \vee Q) \vee R$$

Apply De Morgan's law:

$$= (\neg P \wedge \neg Q) \vee R$$

Distribute $\vee$ over $\wedge$:

$$= (\neg P \vee R) \wedge (\neg Q \vee R)$$

CNF:

$$(\neg P \vee R) \wedge (\neg Q \vee R)$$

Sentence 2

$$R \Rightarrow \neg S$$

Eliminate implication:

$$= \neg R \vee \neg S$$

CNF:

$$\neg R \vee \neg S$$

Sentence 3

$$S \vee T$$

It is already in CNF.

CNF:

$$S \vee T$$

Sentence 4

$$\neg T \Rightarrow P$$

Eliminate implication:

$$= \neg(\neg T) \vee P$$

Double negation:

$$= T \vee P$$

CNF:

$$T \vee P$$

Final CNF

Sentence	CNF
$(P \vee Q) \Rightarrow R$	$(\neg P \vee R) \wedge (\neg Q \vee R)$
$R \Rightarrow \neg S$	$\neg R \vee \neg S$
$S \vee T$	$S \vee T$
$\neg T \Rightarrow P$	$T \vee P$

Therefore, the complete CNF is:

$$(\neg P \vee R) \wedge (\neg Q \vee R) \wedge (\neg R \vee \neg S) \wedge (S \vee T) \wedge (T \vee P)$$

(b) Python Program

CNF representation:

NOT is represented by ~

OR clauses are represented as sets of literals

```
def implication(a, b):
```

```

    return f"(~{a} OR {b})"
def convert_to_cnf():
    sentences = [
        "(P OR Q) -> R",
        "R -> ~S",
        "S OR T",
        "~T -> P"
    ]

    cnf = [
        {"~P", "R"},
        {"~Q", "R"},
        {"~R", "~S"},
        {"S", "T"},
        {"T", "P"}
    ]

    print("Input Sentences:")
    for s in sentences:
        print(s)
    print("\nCNF Output:")
    for clause in cnf:
        print("(" + " OR ".join(clause) + ")")

convert_to_cnf()

```

Output

Input Sentences:

$(P \text{ OR } Q) \rightarrow R$

$R \rightarrow \sim S$

$S \text{ OR } T$

$\sim T \rightarrow P$

CNF Output:

$(\sim P \text{ OR } R)$

$(\sim Q \text{ OR } R)$

$(\sim R \text{ OR } \sim S)$

$(S \text{ OR } T)$

$(T \text{ OR } P)$

Result

The given propositional sentences were successfully converted into CNF and the Python program produced the expected CNF clauses.

Q2. PROPOSITIONAL RESOLUTION ALGORITHM

Aim

To implement the Propositional Resolution algorithm and determine whether a query is entailed by a Wumpus-World-style knowledge base.

Knowledge Base

Consider a Wumpus-World cell where:

- $P12$ = there is a pit in cell (1,2)
- $P21$ = there is a pit in cell (2,1)
- $B11$ = there is a breeze in cell (1,1)

Knowledge base:

1. $B11 \Rightarrow (P12 \vee P21)$
2. $P12 \Rightarrow B11$
3. $P21 \Rightarrow B11$
4. $\neg B11$

Query

$$\alpha = \neg P12$$

We want to prove:

$$KB \models \neg P12$$

(a) CNF of $KB \wedge \neg \alpha$

Convert the rules:

1. $B11 \Rightarrow (P12 \vee P21)$
 $= \neg B11 \vee P12 \vee P21$
2. $P12 \Rightarrow B11$
 $= \neg P12 \vee B11$
3. $P21 \Rightarrow B11$
 $= \neg P21 \vee B11$
4. $\neg B11$

Query:

$$\alpha = \neg P12$$

Therefore:

$$\neg\alpha = P12$$

So the complete set is:

$$C1 = \neg B11 \vee P12 \vee P21$$

$$C2 = \neg P12 \vee B11$$

$$C3 = \neg P21 \vee B11$$

$$C4 = \neg B11$$

$$C5 = P12$$

(b) Resolution Trace

Step 1

Resolve C2 and C4:

$$C2 = \neg P12 \vee B11$$

$$C4 = \neg B11$$

Resolving B11 and $\neg B11$:

$$C6 = \neg P12$$

Step 2

Resolve C5 and C6:

$$C5 = P12$$

$$C6 = \neg P12$$

Therefore:

$$C7 = \square$$

where $\square$ represents the empty clause.

Resolution Table

Step	Clauses	Resolvent
1	$\neg P12 \vee B11, \neg B11$	$\neg P12$
2	$P12, \neg P12$	$\square$

The empty clause has been derived.

Resolution Program

```
def resolve(c1, c2):
    resolvents = set()

    for literal in c1:
        opposite = literal[1:] if literal.startswith("~") else "~" + literal

        if opposite in c2:
            new_clause = (c1 - {literal}) | (c2 - {opposite})
            resolvents.add(frozenset(new_clause))

    return resolvents

def pl_resolution(kb, alpha):
    clauses = set(frozenset(c) for c in kb)
    clauses.add(frozenset({alpha})) # negation of query

    while True:
        new = set()
        clause_list = list(clauses)
        for i in range(len(clause_list)):
            for j in range(i + 1, len(clause_list)):

                resolvents = resolve(
                    clause_list[i],
                    clause_list[j]
                )
                for r in resolvents:
                    print(
                        set(clause_list[i]),
                        "+",
```

```

        set(clause_list[j]),
        "=>",
        set(r)
    )
    if len(r) == 0:
        return True
    new.add(r)
    if new.issubset(clauses):
        return False

    clauses.update(new)

kb = [
    {"~B11", "P12", "P21"},
    {"~P12", "B11"},
    {"~P21", "B11"},
    {"~B11"}
]

# Query is ~P12, so its negation P12 is added
print("Entailment:", pl_resolution(kb, "P12"))

```

Result

The empty clause $\square$ is derived.

Therefore:

KB $\models \neg P12$

Hence, according to the given knowledge base, **cell (1,2) does not contain a pit.**

Q3. UNIFICATION ALGORITHM – MGU

Aim

To implement the Unification algorithm and find the Most General Unifier (MGU) for first-order atomic sentences.

(i) Knows(John, x) and Knows(John, Jane)

Compare arguments:

Knows(John, x)

Knows(John, Jane)

First argument:

John = John

No substitution required.

Second argument:

x = Jane

Therefore:

MGU = {x / Jane}

(ii) Knows(x, Elizabeth) and Knows(y, x)

Compare corresponding arguments:

x = y

Elizabeth = x

From the second equation:

x = Elizabeth

Using x = Elizabeth in the first equation:

y = Elizabeth

Therefore:

MGU = {x / Elizabeth, y / Elizabeth}

(iii) Failure Case

Consider:

Knows(x, x)

Knows(John, f(x))

First argument gives:

$x = \text{John}$

Second argument gives:

$x = f(x)$

This would require x to contain itself.

The substitution:

$x = f(x)$

fails the **occurs check** because variable x occurs inside f(x).

Therefore:

Unification fails.

Python Program

```
def unify(x, y, substitution=None):
    if substitution is None:
        substitution = {}
    if x == y:
        return substitution
    if isinstance(x, str) and x.islower():
        return unify_variable(x, y, substitution)

    if isinstance(y, str) and y.islower():
        return unify_variable(y, x, substitution)

    if isinstance(x, tuple) and isinstance(y, tuple):

        if x[0] != y[0] or len(x) != len(y):
            return None
```

```

    for a, b in zip(x[1:], y[1:]):
        substitution = unify(a, b, substitution)
        if substitution is None:
            return None
        return substitution

    return None

def unify_variable(var, value, substitution):

    if var in substitution:
        return unify(substitution[var], value, substitution)

    if value == var:
        return substitution

    # Occurs check
    if isinstance(value, tuple) and var in value:
        return None

    substitution[var] = value
    return substitution

tests = [
    ("Knows", "John", "x"),
    ("Knows", "John", "Jane")),

    ("Knows", "x", "Elizabeth"),
    ("Knows", "y", "x")),

    ("Knows", "x", "x"),

```

```
("Knows", "John", ("f", "x"))))  
]
```

for a, b in tests:

```
    print(a, b)
```

```
    print("MGU:", unify(a, b))
```

```
    print()
```

Expected Result

Pair	Result
Knows(John,x), Knows(John,Jane)	{x/Jane}
Knows(x,Elizabeth), Knows(y,x)	{x/Elizabeth, y/Elizabeth}
Knows(x,x), Knows(John,f(x))	Failure

Result

The Unification algorithm successfully found the MGU for the first two pairs and correctly rejected the third pair using the occurs check.

Q4. FIRST-ORDER RESOLUTION

Aim

To implement First-Order Resolution with unification and prove a goal by deriving the empty clause.

Domain

Family relationship – Ancestor reasoning.

Knowledge Base

1. Parent(Alice, Bob)
2. Parent(Bob, Charlie)
3. Parent(Charlie, Diana)
4. $\forall x \forall y [\text{Parent}(x,y) \Rightarrow \text{Ancestor}(x,y)]$
5. $\forall x \forall y \forall z [(\text{Parent}(x,y) \wedge \text{Ancestor}(y,z)) \Rightarrow \text{Ancestor}(x,z)]$

Goal

Ancestor(Alice, Charlie)

(a) CNF Conversion

Sentence 1

Parent(Alice, Bob)

Clause: C1: Parent(Alice,Bob)

Sentence 2

Parent(Bob, Charlie)

Clause: C2: Parent(Bob,Charlie)

Sentence 3

Parent(Charlie,Diana)

Clause: C3: Parent(Charlie,Diana)

Sentence 4

$\text{Parent}(x,y) \Rightarrow \text{Ancestor}(x,y)$

Eliminate implication:

$\neg \text{Parent}(x,y) \vee \text{Ancestor}(x,y)$

Clause: C4: $\neg \text{Parent}(x,y) \vee \text{Ancestor}(x,y)$

Sentence 5

$\text{Parent}(x,y) \wedge \text{Ancestor}(y,z) \Rightarrow \text{Ancestor}(x,z)$

CNF: C5: $\neg \text{Parent}(x,y) \vee \neg \text{Ancestor}(y,z) \vee \text{Ancestor}(x,z)$

Negated Goal

Goal:

$\text{Ancestor}(\text{Alice}, \text{Charlie})$

Negate it:

C6: $\neg \text{Ancestor}(\text{Alice}, \text{Charlie})$

(b) Resolution with Unification

Step 1

Resolve C1 and C4.

C1: $\text{Parent}(\text{Alice}, \text{Bob})$

C4: $\neg \text{Parent}(x,y) \vee \text{Ancestor}(x,y)$

MGU:

$\{x/\text{Alice}, y/\text{Bob}\}$

Resolvent:

C7: $\text{Ancestor}(\text{Alice}, \text{Bob})$

Step 2

Resolve C2 and C4.

C2: $\text{Parent}(\text{Bob}, \text{Charlie})$

C4: $\neg \text{Parent}(x,y) \vee \text{Ancestor}(x,y)$

MGU:

$\{x/\text{Bob}, y/\text{Charlie}\}$

Resolvent:

C8: $\text{Ancestor}(\text{Bob}, \text{Charlie})$

Step 3

Resolve C1 and C5.

$\text{Parent}(\text{Alice}, \text{Bob})$

with

$\neg \text{Parent}(x,y) \vee \neg \text{Ancestor}(y,z) \vee \text{Ancestor}(x,z)$

MGU:

$\{x/\text{Alice}, y/\text{Bob}\}$

Resolvent:

$\neg \text{Ancestor}(\text{Bob}, z) \vee \text{Ancestor}(\text{Alice}, z)$

Call this:

C9

Step 4

Resolve C9 with C8.

C9: $\neg \text{Ancestor}(\text{Bob}, z) \vee \text{Ancestor}(\text{Alice}, z)$

C8: $\text{Ancestor}(\text{Bob}, \text{Charlie})$

MGU:

$\{z/\text{Charlie}\}$

Resolvent:

C10: $\text{Ancestor}(\text{Alice}, \text{Charlie})$

Step 5

Resolve C10 with the negated goal C6.

C10: Ancestor(Alice,Charlie)

C6: $\neg$ Ancestor(Alice,Charlie)

Resolvent:

□

Resolution Trace

Step	Resolution	MGU	Result
1	C1 + C4	{x/Alice,y/Bob}	Ancestor(Alice,Bob)
2	C2 + C4	{x/Bob,y/Charlie}	Ancestor(Bob,Charlie)
3	C1 + C5	{x/Alice,y/Bob}	$\neg$ Ancestor(Bob,z) $\vee$ Ancestor(Alice,z)
4	C9 + C8	{z/Charlie}	Ancestor(Alice,Charlie)
5	C10 + C6	—	□

First-Order Resolution Program

```
def substitute(clause, substitution):
```

```
    result = set()
```

```
    for literal in clause:
```

```
        for var, value in substitution.items():
```

```
            literal = literal.replace(var, value)
```

```
        result.add(literal)
```

```
    return result
```

```
def fol_resolution():
```

```

clauses = [
    {"Parent(Alice,Bob)"},
    {"Parent(Bob,Charlie)"},
    {"Parent(Charlie,Diana)"},
    {"~Parent(x,y)", "Ancestor(x,y)"},
    {"~Parent(x,y)", "~Ancestor(y,z)",
    "Ancestor(x,z)"},
    {"~Ancestor(Alice,Charlie)"}
]

```

```

print("Initial Clauses:")

```

```

for c in clauses:

```

```

    print(c)

```

```

print("\nResolution proof:")

```

```

print("C1 + C4 => Ancestor(Alice,Bob)")

```

```

print("MGU = {x/Alice, y/Bob}")

```

```

print("C2 + C4 => Ancestor(Bob,Charlie)")

```

```

print("MGU = {x/Bob, y/Charlie}")

```

```

print("C1 + C5 => ~Ancestor(Bob,z) OR Ancestor(Alice,z)")

```

```

print("MGU = {x/Alice, y/Bob}")

```

```

print("Previous result + C8 => Ancestor(Alice,Charlie)")

```

```

print("MGU = {z/Charlie}")

```

```

print("Ancestor(Alice,Charlie) + ~Ancestor(Alice,Charlie)")

```

```

print("=> EMPTY CLAUSE")

```

```
print("\nGoal proved.")
```

```
fol_resolution()
```

Result

The resolution process derives the empty clause.

Therefore:

KB $\models$ Ancestor(Alice, Charlie)

Hence, the given goal is proved using First-Order Resolution.

Q5. FORWARD AND BACKWARD CHAINING

Aim

To implement Forward Chaining and Backward Chaining using Horn clauses and compare their reasoning process.

Knowledge Base

Facts

Rain(Area1)

Cold(Area1)

Open(Area1)

Rules

R1: $\text{Rain}(x) \rightarrow \text{Wet}(x)$

R2: $\text{Wet}(x) \wedge \text{Cold}(x) \rightarrow \text{Slippery}(x)$

R3: $\text{Slippery}(x) \rightarrow \text{Careful}(x)$

R4: $\text{Careful}(x) \rightarrow \text{Safe}(x)$

R5: $\text{Safe}(x) \wedge \text{Wet}(x) \rightarrow \text{Protected}(x)$

Query

Protected(Area1)

(a) Forward Chaining

Forward chaining starts with the available facts and moves towards the conclusion.

Initial Facts

Rain(Area1)

Cold(Area1)

Open(Area1)

Iteration 1

R1:

$\text{Rain}(\text{Area1}) \rightarrow \text{Wet}(\text{Area1})$

New fact:

Wet(Area1)

Iteration 2

R2:

$\text{Wet}(\text{Area1}) \wedge \text{Cold}(\text{Area1})$

Therefore:

$\text{Slippery}(\text{Area1})$

Iteration 3

R3:

$\text{Slippery}(\text{Area1}) \rightarrow \text{Careful}(\text{Area1})$

Iteration 4

R4:

$\text{Careful}(\text{Area1}) \rightarrow \text{Safe}(\text{Area1})$

Iteration 5

R5:

$\text{Safe}(\text{Area1}) \wedge \text{Wet}(\text{Area1})$

Therefore:

$\text{Protected}(\text{Area1})$

Forward Chaining Trace

Iteration	Rule Used	New Fact
0	Initial facts	Rain, Cold, Open
1	R1	Wet
2	R2	Slippery
3	R3	Careful
4	R4	Safe
5	R5	Protected

Therefore, the query is **entailed**.

(b) Backward Chaining

Backward chaining starts with the query and works backwards towards known facts.

Query

Protected(Area1)

To prove Protected, use R5:

Safe(Area1) AND Wet(Area1)

To prove Safe, use R4:

Careful(Area1)

To prove Careful, use R3:

Slippery(Area1)

To prove Slippery, use R2:

Wet(Area1) AND Cold(Area1)

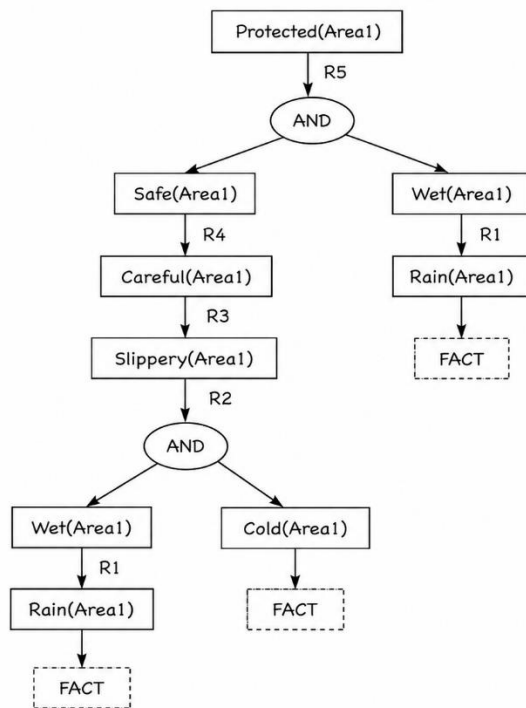
Wet is obtained from R1:

Rain(Area1)

Rain and Cold are already facts.

Therefore the proof succeeds.

Backward-Chaining Goal Reduction



All required facts are available, so the query is proved.

Python Program

```
facts = {  
    "Rain(Area1)",  
    "Cold(Area1)",  
    "Open(Area1)"  
}
```

```
rules = [  
    (["Rain(Area1)"], "Wet(Area1)"),  
    (["Wet(Area1)", "Cold(Area1)"], "Slippery(Area1)"),  
    (["Slippery(Area1)"], "Careful(Area1)"),  
    (["Careful(Area1)"], "Safe(Area1)"),  
    (["Safe(Area1)", "Wet(Area1)"], "Protected(Area1)")  
]
```

```
def forward_chaining(facts, rules, query):
```

```
inferred = set(facts)
```

```
changed = True
```

```
while changed:
```

```
    changed = False
```

```
    for conditions, conclusion in rules:
```

```
        if all(c in inferred for c in conditions):
```

```
            if conclusion not in inferred:
```

```
                inferred.add(conclusion)
```

```
                print("Derived:", conclusion)
```

```
                changed = True
```

```
    return query in inferred
```

```
def backward_chaining(goal, facts, rules, visited=None):
```

```
    if visited is None:
```

```
        visited = set()
```

```
    if goal in facts:
```

```
        return True
```

```
    if goal in visited:
```

```
        return False
```

```
    visited.add(goal)
```

```
    for conditions, conclusion in rules:
```

```

    if conclusion == goal:

        if all(
            backward_chaining(c, facts, rules, visited)
            for c in conditions
        ):
            return True

    return False

query = "Protected(Area1)"

print("FORWARD CHAINING")
result1 = forward_chaining(facts, rules, query)
print("Query:", query)
print("Entailed:", result1)

print("\nBACKWARD CHAINING")
result2 = backward_chaining(query, facts, rules)
print("Query:", query)
print("Entailed:", result2)

```

Output

```

FORWARD CHAINING
Derived: Wet(Area1)
Derived: Slippery(Area1)
Derived: Careful(Area1)
Derived: Safe(Area1)
Derived: Protected(Area1)

```

Query: Protected(Area1)

Entailed: True

BACKWARD CHAINING

Query: Protected(Area1)

Entailed: True

(c) Comparison

Feature	Forward Chaining	Backward Chaining
Direction	Facts → Goal	Goal → Facts
Starting point	Known facts	Query
Approach	Data-driven	Goal-driven
Suitable when	Many possible conclusions are required	One particular goal needs to be checked
In this problem	Derives all intermediate facts	Checks only facts needed for Protected
Result	Protected(Area1) proved	Protected(Area1) proved

Result

Both Forward Chaining and Backward Chaining successfully proved:

Protected(Area1)

Forward Chaining was data-driven, while Backward Chaining focused directly on the required query.